FORE SCHOOL OF MANAGEMENT

A Project Report

on

“ShopAssist-AI: An Agentic RAG Customer-Support and Inventory Assistant for an Online Clothing Store”

(Built in n8n with local AI, RAG, multi-agent design and human-in-the-loop escalation)



Course: Agentic AI for Business Automation

Capstone Project (Individual)

Submitted by

065087: Kritika Bhachawat

PGDM (Business Data Analytics), 2025–27

FORE School of Management, New Delhi

Date of Submission: 10th October 2026

Submitted to: Prof. Shiva Botchu

Contents

#	Section
1	Executive summary
2	Problem statement
3	Objectives
4	Solution: what ShopAssist does
5	System architecture
6	Technology stack (summary)
7	Workflows in detail (01–06)
8	The application: UI / UX and dashboard
9	Questions the system answers
10	Exceptions and how they are handled
11	Testing, evaluation and results
12	Business KPIs
13	Challenges faced and how they were solved
14	Limitations
15	Future recommendations
16	Conclusion
A	Appendix A – Repository structure
B	Appendix B – How to run the project
C	Appendix C – Database schema
D	Appendix D – Agent instructions and business rules
E	Appendix E – Links and references

1. Executive summary

ShopAssist-AI is an AI customer-support assistant for **UrbanThread**, a small (fictional) Indian online clothing store. It is built in **n8n** as six connected workflows and runs on a laptop with a free, local AI model (**Ollama qwen2.5:3b**).

Customers chat with ShopAssist in plain English, or just type one word such as an order ID, "returns" or "diwali". The assistant:

- answers policy and offer questions from the store's own documents using **RAG** (Retrieval-Augmented Generation with **Pinecone**): a FAQ PDF and four web pages;
- looks up live **order status** in a **Postgres** database (Supabase);
- **hands angry customers and refunds above Rs 5,000 to a human**, with a structured ticket and alerts by **Slack, SMS (Twilio) and e-mail (Gmail SMTP)**;
- passes stock and purchasing questions to a second AI agent, the **Inventory Agent** (multi-agent design), which checks stock and drafts **purchase orders** for manager approval;
- sends customers an SMS automatically when the order system reports a **delay** (webhook).

A web app shows the chat next to a **live business dashboard**: orders by status, tickets, low stock and purchase orders. On 14 automatic test questions the final system scored **14 / 14** (average answer time **10 s**). In testing it handled **44 conversations**; **82 %** of them were resolved without a human.

> **Key design idea:** a small local model is cheap and private but not fully reliable. Business rules that must never fail are therefore enforced by deterministic workflow steps around the AI, not left to the model: escalation, the Rs 5,000 refund limit, purchase-order quantities, time limits and fallbacks. This raised the score from 9 / 14 to 14 / 14.

2. Problem statement

UrbanThread sells clothing online across India and has a support team of two people. Most incoming messages are repetitive: "where is my order?", "what is the return window?", "when will I get my refund?", "which size should I buy?", "is there a Diwali code?". This creates five business problems:

1. **Slow first response during sales.** Message volume peaks during festive sales such as Diwali, exactly when customers expect quick answers. Customers wait hours for a reply that could take seconds.
2. **Urgent cases are buried.** Angry customers and high-value refunds wait in the same queue as simple questions, which increases churn and bad reviews.
3. **No structured record or alert.** When a case is escalated, nothing records it in a structured form and nobody is alerted at once.
4. **No proactive delay updates.** Customers are not told when a courier delay happens, so they contact support and the load rises further.
5. **Stock problems are found late.** Low stock is noticed by hand, so popular sizes go out of stock and sales are lost.

Who is affected

Stakeholder	Pain today	What ShopAssist changes
Customer	Waits for simple answers; no update on delays	Instant 24×7 answers; SMS when an order is delayed; a ticket ID when a human is needed
Support team	Repeats the same answers; urgent cases get lost	Only real escalations reach them, already summarized, with priority and sentiment
Store manager	No numbers on support load or stock risk	Live dashboard and KPI report; purchase orders drafted for approval

3. Objectives

1. Answer common questions instantly, 24×7, **only** from the store's own policies and web pages (no invented policies or coupon codes).
2. Let customers check order status with an order ID or a phone number.
3. Detect cases that need a person (anger, a request for a human, a refund above Rs 5,000, no answer found) and hand them over with a structured ticket and instant Slack, SMS and e-mail alerts.
4. Tell customers by SMS automatically when an order is delayed.
5. Add a second domain-expert agent for inventory: answer stock questions, list low stock, draft purchase orders.
6. Measure the result: an automatic evaluation and business KPIs.
7. Spend nothing: local AI (Ollama) and free tiers (Pinecone, Supabase), with no paid API.

4. Solution: what ShopAssist does

The flow below shows one customer message from start to finish.

Step	What happens	Where
1	The customer types a message in the chat (n8n public chat, the web app or the Terminal).	Workflow 03 – Chat Trigger
2	**Detect Intent (rules)** checks the message for clear signals: anger, a request for a human, a refund, stock, offers, off-topic. It adds a short note for the AI and sets flags.	Workflow 03 – Code node
3	The **AI Tools Agent** (qwen2.5:3b) reads the message, the note and the last 10 messages (memory). It decides which tool to call.	Workflow 03 – AI Agent
4	Tools: faq_knowledge / website_knowledge search Pinecone (RAG). order_lookup queries Postgres. inventory_expert asks the Inventory Agent (workflow 06). escalate_to_human calls workflow 04.	Workflows 03, 04, 06
5	The agent writes the answer from the tool results only and names the source.	Workflow 03
6	**Guaranteed escalation:** if a human is needed and the reply has no ticket ID, the workflow creates the ticket itself and adds the ID to the reply.	Workflow 03 → 04
7	Workflow 04 builds a ticket (LLM + structured output), saves it (TKT-000N) and alerts Slack, the customer (SMS) and the support inbox (e-mail).	Workflow 04
8	The dashboard and KPI report read the same database, so managers see conversations, tickets and stock live.	Web app / kpi_report.py

5. System architecture

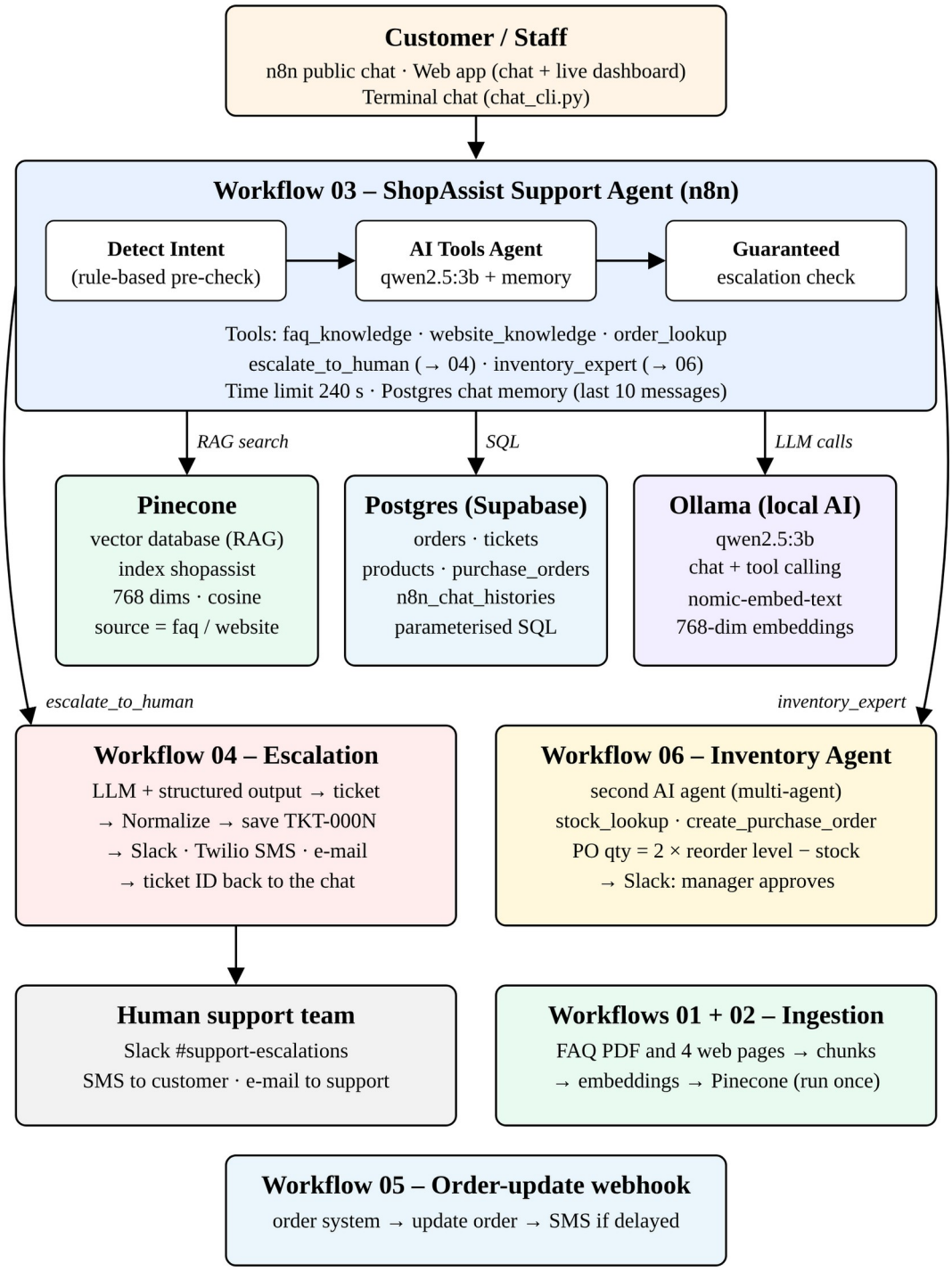


Figure 1 – ShopAssist-AI architecture: six n8n workflows, local AI, Pinecone and Postgres

Deployment: n8n runs in Docker on the laptop (http://localhost:5679), and Ollama runs on the same laptop. Pinecone (vector database) and Supabase (Postgres) use free cloud tiers. The web app is a small Python server (http://localhost:8501). It calls the published n8n chat webhook and reads the database for the dashboard.

6. Technology stack (summary)

Each technology is explained in detail, with what it is, why it was chosen and how it is configured, in the separate document **"ShopAssist-AI – Technology Stack Guide"**.

Layer	Technology	Used for
Workflow automation	n8n 2.x (self-hosted, Docker)	All six workflows, AI agents, webhooks, integrations
AI agent framework	n8n AI Agent (Tools Agent, LangChain-based)	Reasoning + tool calling in workflows 03 and 06
Large language model	Ollama – qwen2.5:3b (local)	Chat answers, tool choice, page summaries, ticket JSON
Embeddings	Ollama – nomic-embed-text (768 dims)	Turning text into vectors for search
Vector database	Pinecone (index shopassist, cosine)	RAG: FAQ and website knowledge with metadata filters
Relational database	Postgres on Supabase	Orders, tickets, products, purchase orders, chat memory
Document processing	Default Data Loader, Recursive Text Splitter, HTML Extract	PDF loading, chunking 1000/200, web-page cleaning
Structured output	Structured Output Parser + JSON schema	Reliable ticket fields (issue type, priority, sentiment)
Notifications	Slack API, Twilio SMS, Gmail SMTP	Escalation alerts, delay SMS, PO approval message
Interfaces	n8n hosted chat, web app (Python + HTML/JS), Terminal CLI	Customer and staff access, live dashboard
Tooling	Python 3 scripts, pg8000, Docker, Git/GitHub	Build, import, diagnose, evaluate, KPIs, version control

7. Workflows in detail

All six workflows are generated from scripts/build_workflows.py and loaded into n8n with scripts/import_to_n8n.py, so they can be recreated exactly at any time. Each canvas carries sticky notes that explain the workflow.

7.1 Workflow 01 – Ingest FAQ PDF into Pinecone

Trigger: manual (run once). **Purpose:** load the store FAQ and policies into the knowledge base.

Download store_faq.pdf from GitHub → **Set MIME type to PDF**. GitHub serves the file as octet-stream, so the type is corrected before loading. → **Default Data Loader** (PDF, metadata source = faq) → **Recursive Character Text Splitter** (1,000 characters, 200 overlap) → **Ollama embeddings** (nomic-embed-text, 768 dims) → **Pinecone insert**.

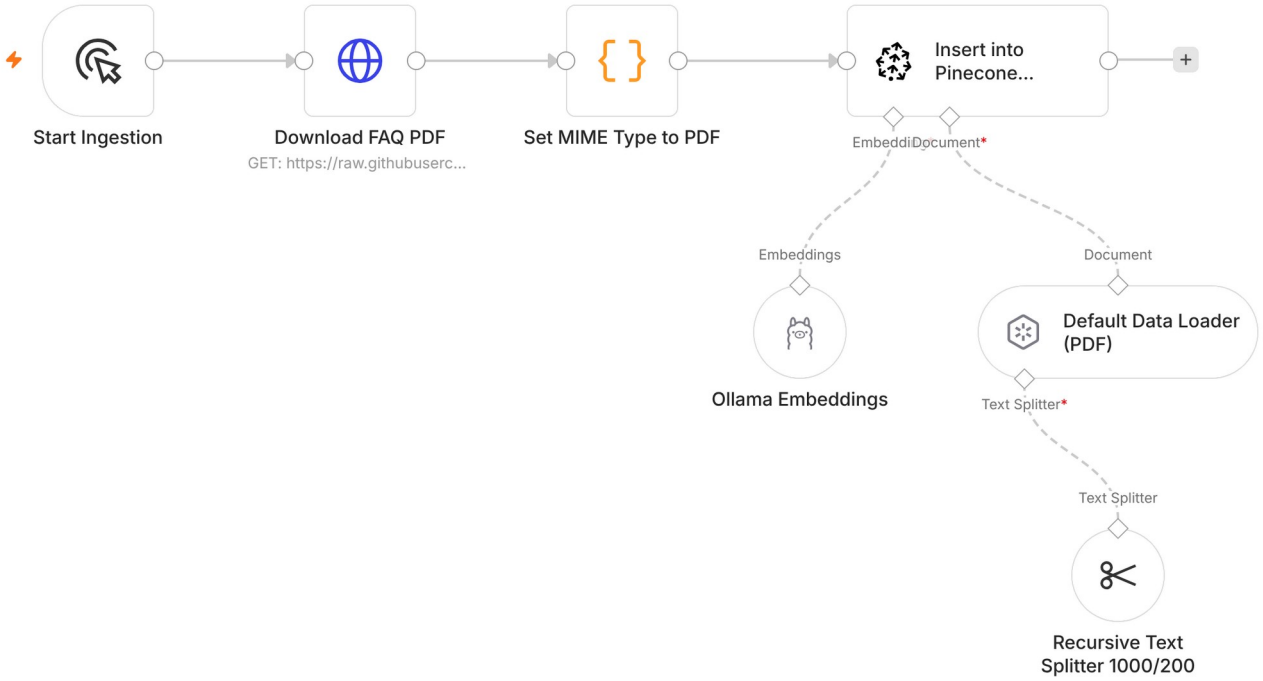


Figure 2 – Workflow 01 canvas

7.2 Workflow 02 – Scrape web pages, summarize and store

Trigger: manual. **Purpose:** add the store's web pages (shipping, size guide, offers, care and sustainability) as a second knowledge base.

Read urls.txt → one item per URL → **HTTP Request** fetches each page → **HTML Extract** keeps the body text → **Basic LLM Chain** (qwen2.5:3b, temperature 0) writes factual bullet summaries → add url and title → embed → Pinecone with metadata source = website. Summaries are shorter and cleaner than raw HTML, so search works better and the agent can cite the page.

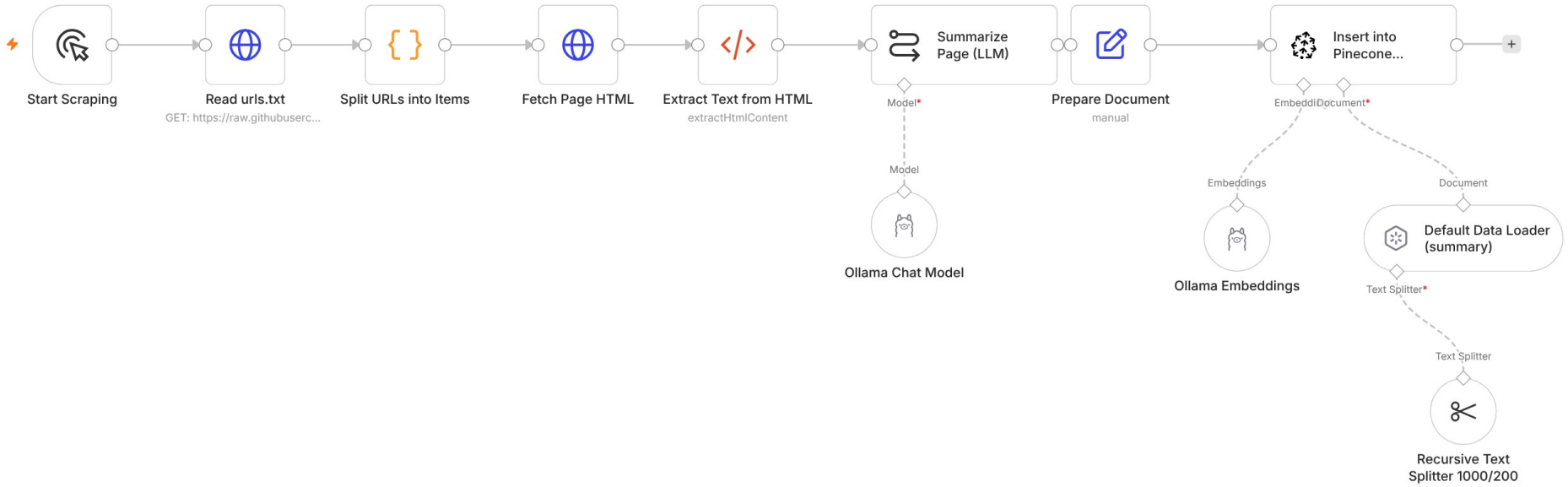


Figure 3 – Workflow 02 canvas

7.3 Workflow 03 – ShopAssist Support Agent (chat)

Trigger: public chat (n8n hosted chat / webhook). **Purpose:** the customer-facing AI agent.

- **Detect Intent (rules):** a JavaScript Code node that adds instructions for the model and sets flags (escalate, refund, order ID, phone, name).
- **ShopAssist AI Agent** (Tools Agent, max 8 steps) with **Postgres Chat Memory** (last 10 messages per session) and five tools:
  - faq_knowledge and website_knowledge: Pinecone retrieval with a metadata filter (agentic RAG, because the agent chooses which knowledge base to search);
  - order_lookup: parameterised Postgres query by order ID or phone;
  - escalate_to_human: calls workflow 04 (sub-workflow tool);
  - inventory_expert: calls the Inventory Agent, workflow 06 (multi-agent).
- **Guardrail after the agent:** Order Details (Postgres) → Needs a Ticket? (rules) → IF Ticket Missing → Create Ticket (workflow 04) → reply with the ticket ID.

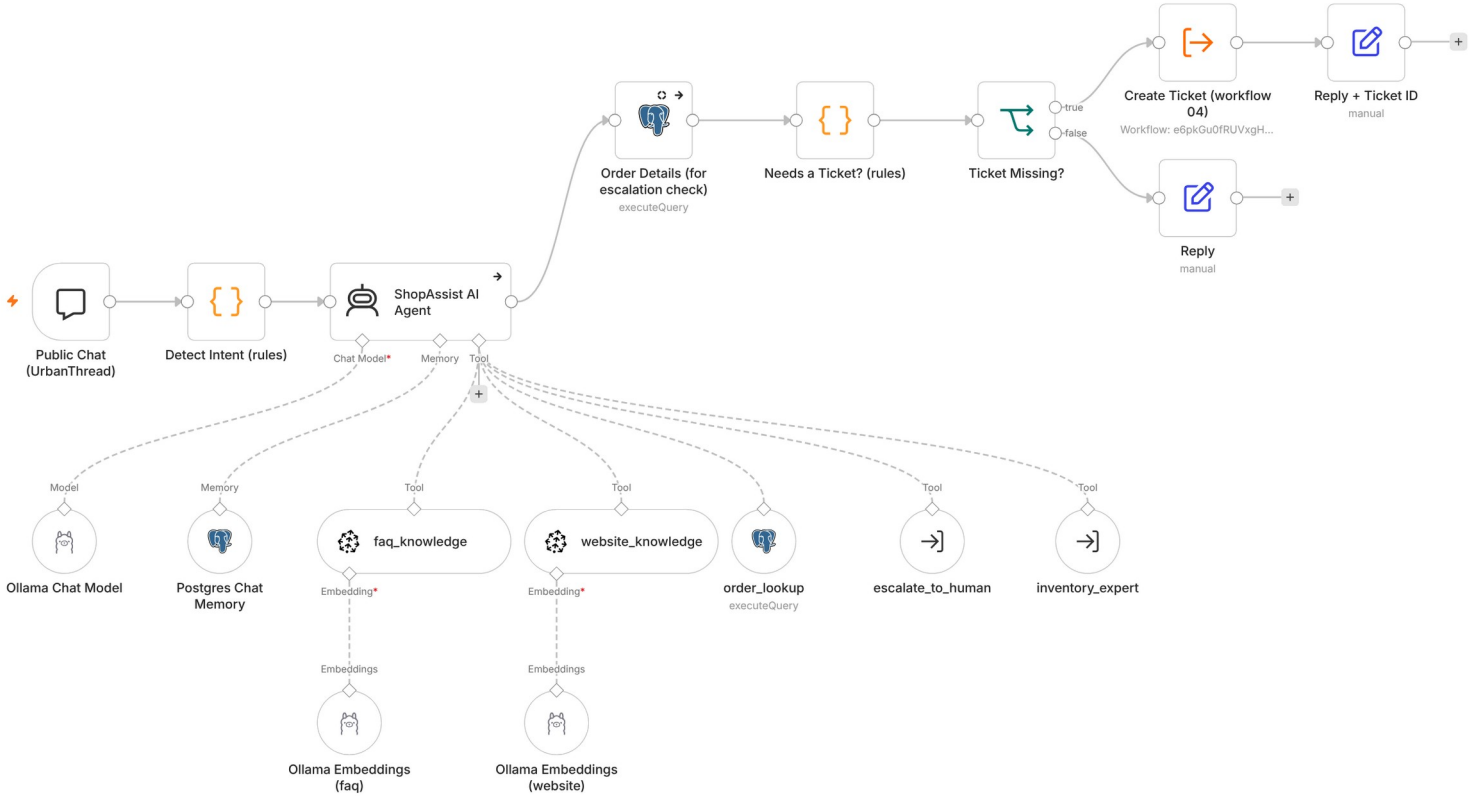


Figure 4 – Workflow 03 canvas: rules → agent with tools → guaranteed escalation

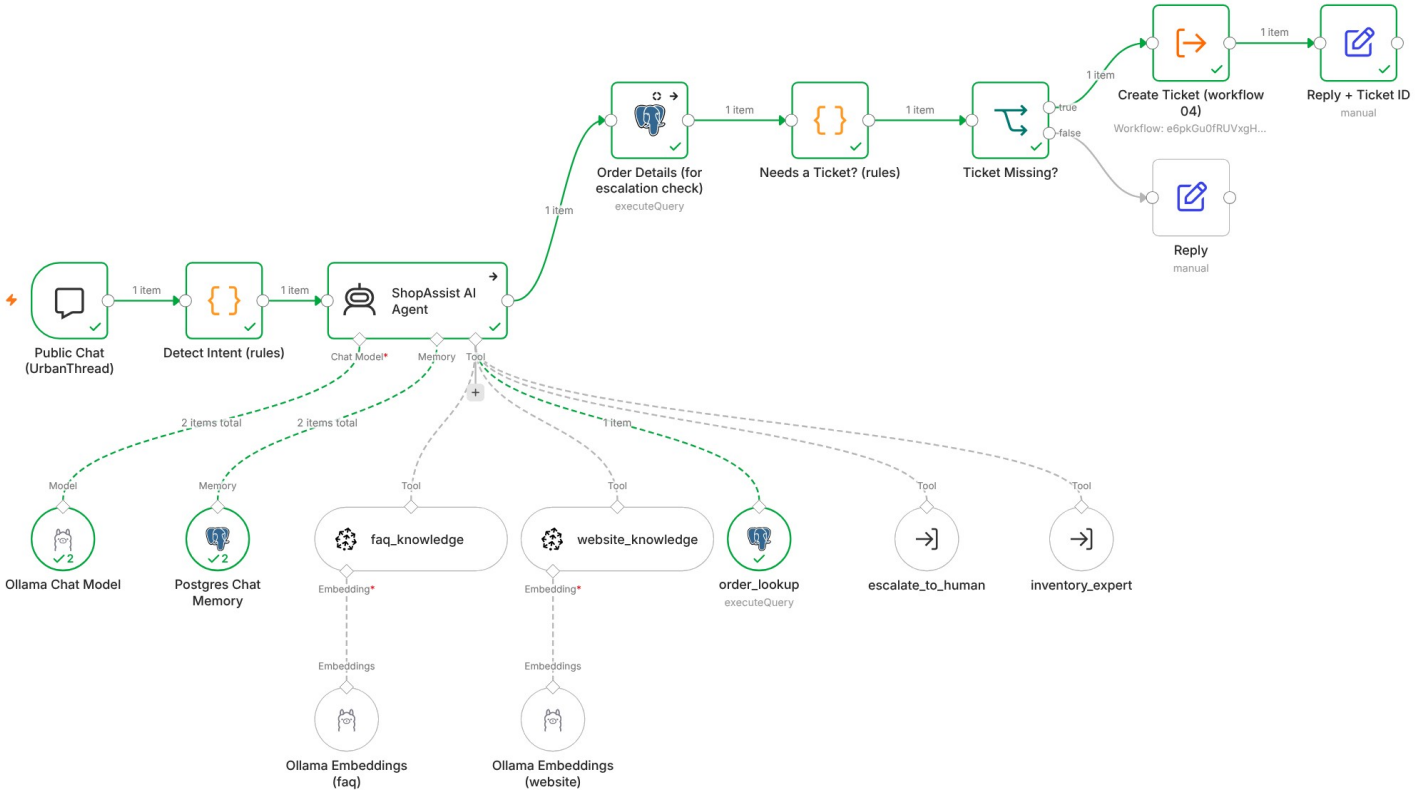


Figure 5 – An execution of workflow 03: an angry customer; the green path shows the order lookup and the guaranteed ticket

7.4 Workflow 04 – Escalation sub-workflow

Trigger: called by workflow 03 with customer name, phone, order ID, reason and conversation summary.

Build Ticket: a Basic LLM Chain with a Structured Output Parser and a JSON schema. → **Normalize Ticket (fallback)** fills any missing field, clamps values to the allowed lists and turns a 10-digit phone number into +91 format. → **Save Ticket in Postgres** (returns TKT-000N) → **Slack** alert to #support-escalations → **Twilio SMS** to the customer → **SMTP e-mail** to the support team → return the ticket ID to the agent. The three notification steps continue on error, so a missing phone number never breaks the chat.

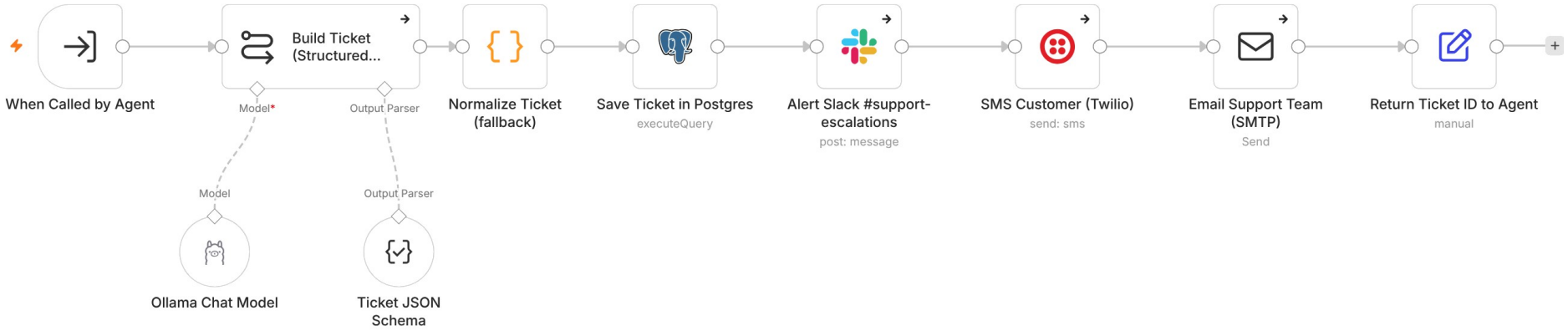


Figure 6 – Workflow 04 canvas

7.5 Workflow 05 – Order status update webhook

Trigger: POST /webhook/order-update from the order or courier system, for example {"order_id":"UT1002","status":"delayed","new_expected_delivery":"2026-10-15"}.

Update the order in Postgres → **IF** status is delayed → **Twilio SMS** to the customer → reply to the caller. An unknown order ID returns ok: false.

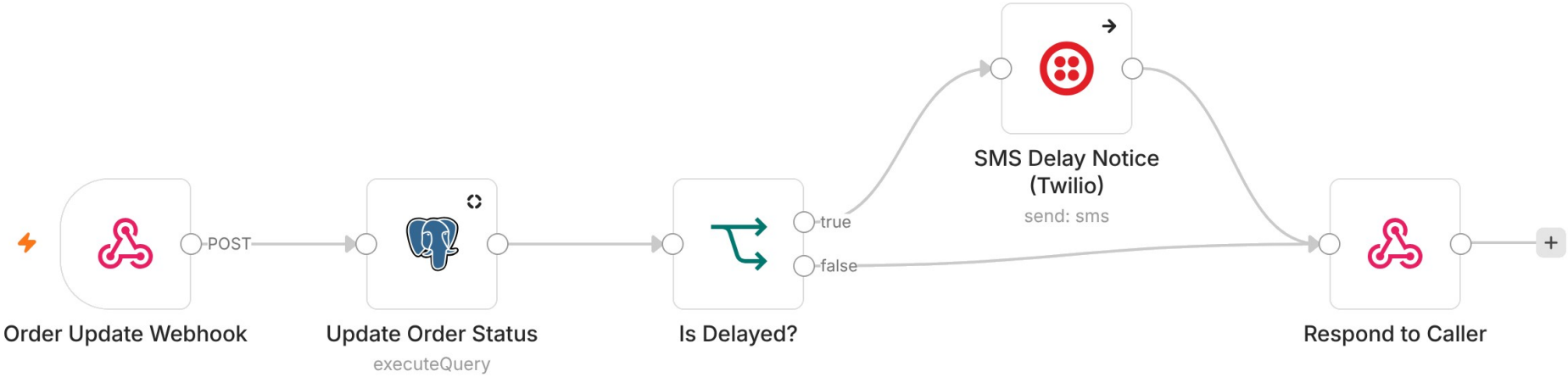


Figure 7 – Workflow 05 canvas

7.6 Workflow 06 – Inventory Agent (second domain expert)

Trigger: called by the support agent through the inventory_expert tool. **Purpose:** stock and procurement expertise.

Inventory AI Agent (max 4 steps) with two Postgres tools: stock_lookup searches the products table by SKU, name, category or size, or with "low" for items at or below reorder level. create_purchase_order drafts a PO in purchase_orders with status pending_approval. The SQL enforces the business rule: quantity = 2 × reorder level − stock, rounded up to a multiple of 5. A **Slack** message then asks the manager to approve.

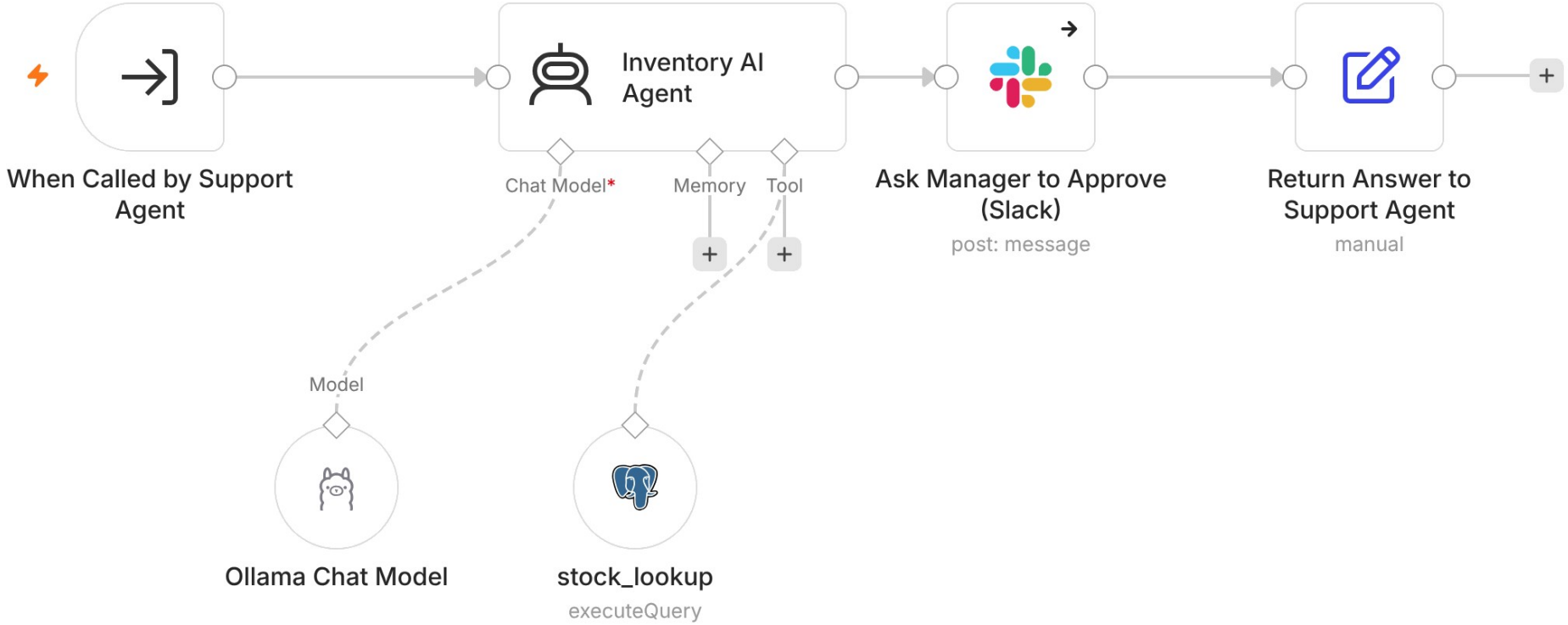


Figure 8 – Workflow 06 canvas

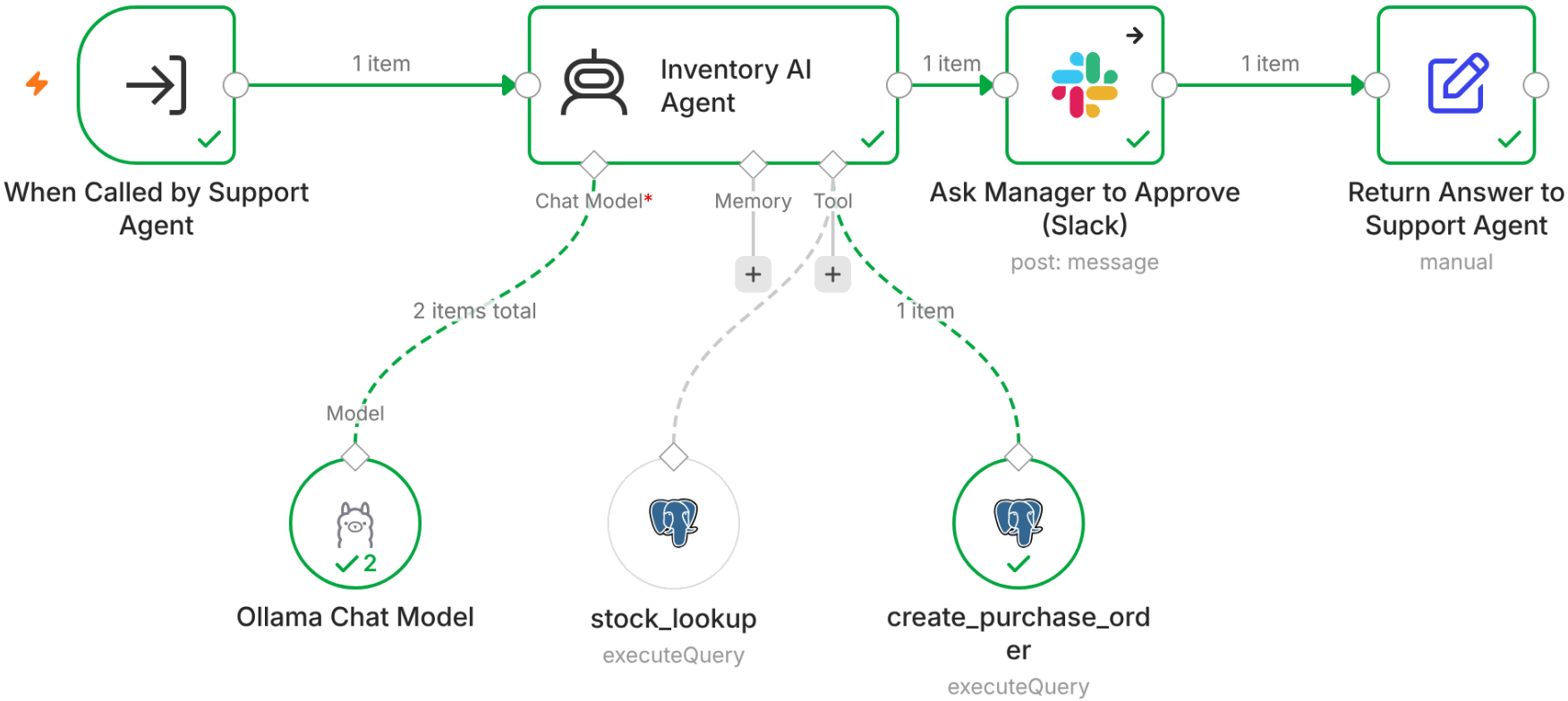


Figure 9 – An execution of workflow 06: the agent called stock_lookup and create_purchase_order, then asked for approval in Slack

8. The application: UI / UX and dashboard

Three ways to talk to ShopAssist, all backed by the same published workflow 03:

Interface	How to open	Best for
Web app + live dashboard	python3 app/server.py → http://localhost:8501	Demo, managers, customers
n8n hosted chat	Workflow 03 → Open chat, or the public chat URL	Showing the n8n execution log
Terminal chat	python3 scripts/chat_cli.py	Quick testing; works even if the browser chat fails

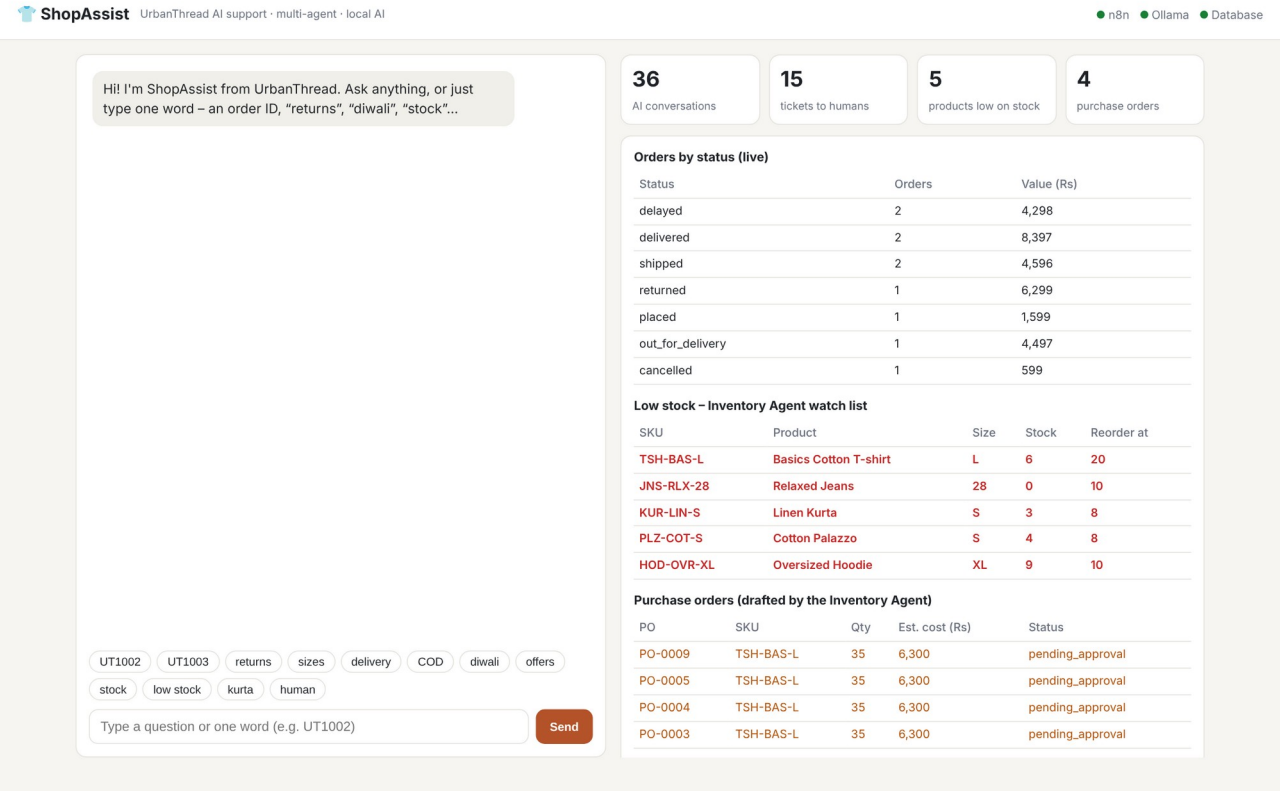


Figure 10 – ShopAssist web app: chat on the left, live business dashboard on the right (test environment data)

Design choices (UI / UX)

- **Two panels:** the conversation on the left and the business view on the right, so a manager sees the effect of each chat at once (new ticket, new purchase order).
- **One-tap chips** (UT1002, returns, sizes, delivery, COD, diwali, offers, stock, low stock, kurta, human): customers can use the assistant without typing a full sentence.
- **Status lights** (n8n, Ollama, Database) at the top right: if something is down, the user sees it immediately instead of a silent failure.
- **KPI cards:** AI conversations, tickets to humans, products low on stock, purchase orders. They refresh every 10 seconds.
- **Colour for risk:** low-stock rows are shown in red and draft purchase orders in amber, so problems stand out.
- **Plain, accessible styling:** large readable text, high contrast and no login needed for the local demo. It uses only the Python standard library, so it is easy to run.

9. Questions the system answers

Category	Example question	Tool / path used	What the customer gets
Store policy (FAQ)	What is your return policy?	faq_knowledge (Pinecone, source=faq)	15-day return policy with conditions
Offers & website	Is there any Diwali discount code?	website_knowledge (source=website)	DIWALI25 – 25% off over Rs 2,499, dates
Order status	Where is my order UT1002?	order_lookup (Postgres)	Status, items, courier, tracking, delivery date
Combined	UT1005 does not fit – can I return it and when is my refund?	order_lookup + faq_knowledge	Order details + return/refund policy
Missing information	When will my order arrive?	Agent asks back	Request for order ID or phone number
Angry customer	Third time asking! UT1003 is late, I want a manager.	Detect Intent → escalate_to_human / guaranteed ticket	Apology + ticket ID; Slack, SMS, e-mail sent
High-value refund	Refund my order UT1004 now (Rs 7,499)	order_lookup → amount > 5,000 → workflow 04	Ticket ID; refund approved by a human
Off-topic	Write a Python script to sort a list.	Detect Intent (out of scope)	Polite refusal, no code
One word	UT1003 / returns / diwali / sizes / COD	Short-message rules in the prompt	The matching answer
Stock (customer)	Is the linen kurta available in size M?	inventory_expert → stock_lookup	In stock / not in stock
Stock (manager)	Which products are low on stock?	inventory_expert → stock_lookup("low")	List of SKUs at or below reorder level
Purchasing	Restock TSH-BAS-L and create a purchase order.	inventory_expert → create_purchase_order	PO-000N draft (35 units), Slack approval
Order system	POST order-update: UT1002 delayed	Workflow 05 webhook	Database updated + SMS to customer

10. Exceptions and how they are handled

A support assistant is only useful if it behaves safely when something unexpected happens. The table lists each exception that was designed for, and tested.

Situation (exception)	How ShopAssist handles it
Customer gives no order ID	Agent asks for the order ID or the registered phone number; it never guesses.
Order ID does not exist	The query returns no rows; the agent says it cannot find the order and asks the customer to check it.
Customer is angry or asks for a human	Detect Intent flags it; the agent calls escalate_to_human. If the model forgets, the guaranteed escalation step creates the ticket anyway.
Refund above Rs 5,000	The amount is read from Postgres (not from the model). Above Rs 5,000 a human ticket is always created. Below it, no ticket (tested with UT1005, Rs 4,497).
Model already created a ticket	The reply is checked for "TKT-"; no duplicate ticket is created.
Question outside the store (code, weather, cricket)	Rule-based note → polite refusal; the agent never writes code.
Risk of an invented coupon code	Offers must come from website_knowledge; the prompt forbids inventing codes.
Model returns bad or incomplete ticket JSON	Normalize Ticket fills missing fields from the inputs and clamps to allowed values; the ticket is always saved.
Model leaves out a tool input	Every sub-workflow tool input has a default value, so the tool call cannot fail on a missing field.
AI step fails completely	The agent node continues on error; the workflow answers from the database (order status) or with a helpful fallback message, and still escalates if needed.
Model is slow or stuck	Time limits per workflow (03: 240 s, 04/06: 150 s), Inventory Agent max 4 steps, and an explicit "do not retry" rule.
Slack / Twilio / e-mail unavailable	Notification nodes continue on error; the ticket is still saved and the customer still gets the ticket ID.
Webhook for an unknown order	Returns ok: false instead of failing.
SQL injection attempts	All queries are parameterised ($1, $2); model text is never pasted into SQL.
Wrong purchase-order quantity	SQL never orders fewer units than the reorder rule; the PO is a draft that needs manager approval.
Date shown one day early (time zone)	Dates are returned as plain text from Postgres.
n8n chat posts to the wrong port (Docker)	WEBHOOK_URL is set with fix_webhook_url.sh; the web app and Terminal chat work regardless.

11. Testing, evaluation and results

python3 scripts/evaluate.py sends 14 test questions to the live chat and checks each answer for the expected facts, for example "15 days", "DIWALI25", "TKT-" or "PO-". It writes the results to docs/evaluation_results.md. The two webhook tests are run with scripts/test_chat.py.

> **Final score: 14 / 14 passed, average answer time 10 s** (local qwen2.5:3b on a MacBook Air). First run before the guardrails: 9 / 14.

#	Test	Expected	Actual answer (shortened)	Time	Result
1	FAQ – return policy	15 days	return items within 15 days from the date of delivery…	23 s	Pass
2	Website – Diwali code	DIWALI25	DIWALI25 – 25% off orders over Rs 2,499 from 20 October…	10 s	Pass
3	Order lookup UT1002	status + courier	UT1002 is Shipped. 1 Linen Kurta (S) …	5 s	Pass
4	Combined UT1005	15 days + refund	mentions return window and refund*	14 s	Pass*
5	Missing order ID	asks for ID	Could you please provide your order ID or phone number?	3 s	Pass
6	Angry customer UT1003	ticket ID	order delayed (Ekart) + ticket TKT-0007	13 s	Pass
7	Out of scope	refusal, no code	only help with UrbanThread orders, returns, sizes…	2 s	Pass
8	Refund > Rs 5,000 (UT1004)	escalation	refund exceeds Rs 5000 → ticket TKT-0008	12 s	Pass
9	One word: UT1003	order status	UT1003 is delayed…	7 s	Pass
10	One word: returns	15 days	processed within 15 days from delivery…	16 s	Pass
11	One word: diwali	DIWALI25	Diwali offers 20 Oct – 5 Nov 2026, DIWALI25…	14 s	Pass
12	Kurta size M in stock?	in stock	The linen kurta is in stock in size M.	5 s	Pass
13	Low stock (manager)	low SKUs	products at or below reorder level: Basics T-shirt (L) 6…	12 s	Pass
14	Create purchase order	PO-000N	purchase order PO-0002 created…*	7 s	Pass*

* Passed the automatic keyword check, but a human reading found a wrong detail. Test 4 added an unrelated "out of stock" remark. Test 14 used 10 units and said "restocked". Both were fixed afterwards: return questions are now routed to order_lookup + faq_knowledge, and the SQL enforces the PO quantity rule (35 units in this case, verified). The agent must now call the PO a draft awaiting approval.

Observations

- **Speed:** 2–23 s per answer on a laptop. Knowledge-base answers are slowest (embedding + vector search + a longer answer). Database answers take 3–5 s.

- **Guardrails made the difference:** the failures on the first run came from the small model saying it had escalated without doing it, inventing a coupon code, and timing out. Rules and fallbacks fixed these; a longer prompt did not.
- **Keyword scoring is not enough:** reading the answers found details that the automatic check missed, so both checks are needed.
- **Multi-agent works:** the support agent passed stock questions to the Inventory Agent, which queried the products table and drafted purchase orders.

12. Business KPIs

Generated by python3 scripts/kpi_report.py from the live Supabase database at the end of testing. The same numbers appear on the web dashboard.

KPI	Value	Meaning
Conversations handled by the AI	44 (48 messages)	Volume taken off the human team
Escalated to a human (tickets)	8 – all high priority	Only urgent cases reached staff
Resolved without a human	≈ 82 %	About 4 in 5 conversations needed no staff time
Products at or below reorder level	5 of 12	Stock risk visible before sell-out
Purchase orders drafted	2 (pending approval)	Procurement started automatically, approved by a human

Tickets by type	Priority	Count
refund	high	6
delivery	high	2

Low-stock SKU	Product	Stock	Reorder level
TSH-BAS-L	Basics Cotton T-shirt (L)	6	20
JNS-RLX-28	Relaxed Jeans (28)	0	10
KUR-LIN-S	Linen Kurta (S)	3	8
PLZ-COT-S	Cotton Palazzo (S)	4	8
HOD-OVR-XL	Oversized Hoodie (XL)	9	10

Orders in the database: 10, worth Rs 30,285 (3 shipped, 2 delivered, 1 delayed, 1 out for delivery, 1 placed, 1 returned, 1 cancelled). The numbers come from testing with fictional data, not from real customers.

13. Challenges faced and how they were solved

Challenge	Cause	Solution
Google Gemini blocked ("403 project denied", model unavailable to new users)	Account / region restrictions on the free API	Switched to a fully local stack with Ollama (qwen2.5:3b + nomic-embed-text): free, private, no API key
Pinecone "vector dimension 0 vs 3072"	Failed Gemini embeddings; index sized for Gemini	Recreated the index at 768 dimensions for nomic-embed-text (setup_pinecone.py)
PDF loader "MIME type doesn't match"	GitHub serves PDFs as application/octet-stream	Added a "Set MIME Type to PDF" step
Scraped text had words glued together	HTML clean-up option removed spaces	Turned clean-up off in HTML Extract
n8n chat "Failed to receive response"	Docker maps port 5679→5678 without WEBHOOK_URL	Set WEBHOOK_URL with a compose override (fix_webhook_url.sh)
Database "tenant/user not found"	Wrong Supabase pooler host/region in .env	Copied host and user from the n8n credential; doctor.py explains the error
Small model skipped escalation / invented codes	3B-parameter model does not always follow instructions	Rule-based pre-check, guaranteed escalation, normalisation, defaults, fallbacks
Inventory questions timed out and blocked other questions	Tool retry loops; one local model serves one request at a time	Time limits on every workflow, max 4 steps, no-retry rule, stuck-run check in doctor

14. Limitations

- Fictional data and a small knowledge base (1 PDF, 4 pages); a real store needs regular re-ingestion. Re-running ingestion creates duplicate vectors.
- The public chat has no login: anyone who knows an order ID can see its status. Production needs OTP verification.
- A 3B local model is slower and less accurate than large cloud models, and answer speed depends on the laptop.
- Anger detection uses keywords plus the model, so it may miss sarcasm or Hinglish.
- Twilio trial accounts can send SMS only to verified numbers. Slack, SMS and e-mail use placeholder credentials until real ones are added.
- The automatic evaluation checks for expected facts, not full answer quality.
- English only; runs only while the laptop, n8n and Ollama are on.

15. Future recommendations

1. **One-click PO approval:** a Slack button that approves the purchase order, updates its status and messages the supplier.
2. **WhatsApp channel** through the Twilio WhatsApp API, with Hindi and Hinglish support.
3. **Customer verification** (OTP to the registered phone) before showing order details.
4. **Larger or hosted model** (for example qwen2.5:7b/14b, or a cloud model) for better wording and fewer wrong details; compare cost and speed.

5. **Scheduled re-scraping and upsert by document ID**, so the knowledge base stays current without duplicates.
6. **LLM-as-judge evaluation** in addition to keyword checks, plus a larger test set run on every change.
7. **Deployment** on a small server or n8n Cloud with a public HTTPS URL, monitoring and alerting.
8. **Richer analytics:** resolution time, sentiment trend, top question types, and a forecast for reordering.

16. Conclusion

ShopAssist-AI shows how n8n can combine **two domain-expert AI agents** (customer support and inventory), **two RAG knowledge bases**, a **SQL database** and a **human-in-the-loop** escalation path into one working business automation. Routine questions are answered in seconds, 24×7, from the store's own information. Urgent and high-value cases reach a person at once through Slack, SMS and e-mail, with a structured ticket. Managers see the effect live on a dashboard.

The most important lesson is that **reliability comes from design, not only from the model**. A small, free, local model became dependable (14 / 14 on the test set) once the business-critical rules were enforced by the workflow itself.

Appendix A – Repository structure

Folder / file	Contents
workflows/	The six n8n workflow JSON files (generated)
scripts/build_workflows.py	Generates all workflow JSON (single source of truth)
scripts/import_to_n8n.py	Creates/updates workflows in n8n via the API, links 03→04/06, assigns credentials
scripts/doctor.py, verify_setup.py	Read-only health checks (code version, .env, Ollama, database, Pinecone, n8n, recent errors)
scripts/setup_database.py, setup_pinecone.py	Create tables + seed data; create the 768-dim Pinecone index
scripts/evaluate.py, kpi_report.py	14-question evaluation; business KPI report
scripts/chat_cli.py, test_chat.py	Terminal chat; end-to-end smoke tests
app/server.py, app/index.html	Web app: chat + live dashboard (port 8501)
prompts/	Agent system prompt, Inventory Agent prompt, ticket JSON schema
database/schema.sql, seed.sql	Tables and fictional data (10 orders, 12 products)
data/	FAQ (Markdown + PDF), web pages (HTML), urls.txt
docs/	Runbook, report, test questions, course mapping, screenshots

Appendix B – How to run the project

Requirements: Docker with n8n, Ollama (ollama pull qwen2.5:3b and ollama pull nomic-embed-text), Python 3 with pip3 install pg8000, free Pinecone and Supabase accounts. Fill in .env from .env.example.

1. python3 scripts/setup_database.py: create the tables and fictional data.
2. python3 scripts/setup_pinecone.py: create the 768-dimension index.
3. python3 scripts/import_to_n8n.py: load all six workflows into n8n.
4. python3 scripts/doctor.py: check everything; follow its WHAT TO DO list.
5. In n8n run workflows 01 and 02 once (fill the knowledge base), then publish 04, 06, 03 and 05.
6. python3 app/server.py → http://localhost:8501, or python3 scripts/chat_cli.py.
7. python3 scripts/evaluate.py and python3 scripts/kpi_report.py for results.

The full step-by-step guide is in docs/RUNBOOK.md.

Appendix C – Database schema

Table	Key columns	Written by
orders	order_id (UT1001…), customer_name, phone, items, amount_inr, payment_method, status, expected_delivery, courier, tracking_no	Seed data; workflow 05 updates status
tickets	ticket_id (TKT-000N, generated), customer_name, phone, order_id, issue_type, sentiment, priority, summary, suggested_action, status	Workflow 04
products	sku, name, category, size, price_inr, stock, reorder_level, supplier, unit_cost_inr	Seed data
purchase_orders	po_id (PO-000N, generated), sku, quantity, supplier, est_cost_inr, reason, status (pending_approval)	Workflow 06
n8n_chat_histories	session_id, message (JSON)	Workflow 03 memory

Appendix D – Agent instructions and business rules

- Answer **only** from tool results; never invent policies, prices, dates or coupon codes. Name the source (FAQ, website page, order database).
- Ask for the order ID or phone number when it is needed and missing.
- Escalate when the customer is angry or asks for a human, when a refund is above Rs 5,000, or when the answer is not in the knowledge bases. Always give the ticket ID.
- Refuse questions that are not about UrbanThread, politely.
- Stock, availability and purchase questions go to the Inventory Agent.
- Purchase order quantity = 2 × reorder level − current stock, rounded up to a multiple of 5. Every PO is a draft until a manager approves it.
- Short messages: an order ID → order lookup; "returns", "refund", "sizes", "COD" → FAQ; "diwali", "offers", "coupon" → website; "stock", a product name → Inventory Agent; "human" → escalation.

Full prompts: prompts/agent_system_prompt.md and prompts/inventory_agent_prompt.md.

Appendix E – Links and references

Project

- GitHub repository – https://github.com/kritikab01/ShopAssist-AI

Tools and platforms

- n8n documentation – https://docs.n8n.io
- n8n AI Agent node – https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/
- Ollama – https://ollama.com
- Qwen2.5 model (Ollama library) – https://ollama.com/library/qwen2.5
- nomic-embed-text (Ollama library) – https://ollama.com/library/nomic-embed-text
- Pinecone documentation – https://docs.pinecone.io
- Supabase documentation – https://supabase.com/docs
- LangChain JS (basis of n8n AI nodes) – https://js.langchain.com
- Slack API – https://api.slack.com
- Twilio SMS documentation – https://www.twilio.com/docs/sms
- Docker documentation – https://docs.docker.com

Research papers

- Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks – https://arxiv.org/abs/2005.11401
- Yao et al. (2022), ReAct: Synergizing Reasoning and Acting in Language Models – https://arxiv.org/abs/2210.03629
- Qwen Team (2024), Qwen2.5 Technical Report – https://arxiv.org/abs/2412.15115
- Nussbaum et al. (2024), Nomic Embed: Training a Reproducible Long Context Text Embedder – https://arxiv.org/abs/2402.01613